

178bNsA BtCQzYK02sfhkis0xkDst wPadT3Y YMPZWQ (English translation)

178bNsA BtCQzYK02sfhkis0xkDst wPadT3Y YMPZWQ - Egidio Romano, Publisher not stated.

- Published: date not stated
- Original: <https://docs.google.com/presentation/d/178bNsA_BtCQzYK02sfhkis0xkDst-wPadT3Y_YMPZWQ>
- Current location: <https://doc-08-8k-slides.googleusercontent.com/export/8o20gl0mgtl0fnnbn5ic4njno4/8noe2uhokgfobqfb7nti3lujhg/1790900100000/100503688158085955534/*/178bNsA_BtCQzYK02sfhkis0xkDst-wPadT3Y_YMPZWQ?exportFormat=pdf>
- Preserved from: https://doc-08-8k-slides.googleusercontent.com/export/8o20gl0mgtl0fnnbn5ic4njno4/8noe2uhokgfobqfb7nti3lujhg/1790900100000/100503688158085955534/*/178bNsA_BtCQzYK02sfhkis0xkDst-wPadT3Y_YMPZWQ?exportFormat=pdf (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content (translated into English)

Machine translation of 178bnsa-btcqzyk02sfhkis0xkdst-wpadt3y-ympzwq.md, which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

\$ whoami

- Egidio Romano (aka EgiX)
- Web Application Security Researcher (2007 - ∞)
- Security Specialist @ Secunia (2013 - 2014)
- Security Consultant @ Minded Security (2014 - 2015)
- Freelance IT Security Consultant (2016 - Present)

<https://karmainsecurity.com> \$ agenda

- What is Discuz!
- Chaining multiple bugs:

- RCE with administrator privileges
- CAPTCHA bypass using AI
- Race Condition leading to Authentication Bypass
 - Brief exploit analysis & Disclosure Timeline
 - Live hacking demo: exploiting Discuz! X5.0

What is Discuz!

Physical HTML, Computing CSS,JS

<https://en.wikipedia.org/wiki/Discuz!> What is Discuz! Key features of Discuz!

● Piattaforma per community e forum: un ecosistema completo per creare siti web con discussioni e post tematici in modo strutturato. Physical HTML, ● Tecnologia: scritto principalmente in linguaggio Computing CSS,JS PHP, supporta database MySQL e PostgreSQL. ● Popolarità: nel 2010 venivano stimati circa 1,4 milioni di siti web realizzati con Discuz! in tutto il mondo, in particolare in Cina, dove è più diffuso anche oggi. Cos'è Discuz! Discuz! X5.0, rilasciato ufficialmente il 20 marzo 2026, rappresenta l'ultima generazione della piattaforma. Progettato per rispondere alle esigenze del web moderno, introduce un'architettura profondamente rinnovata e richiede PHP 8.0+.

Key new features of version X5.0: ● New template system with a fully responsive interface optimized for mobile devices. Physical HTML, ● Computing Native internationalization for simplified management of CSS,JS multilingual environments. ● Automatic updates for faster and more effective distribution of security patches. ● Advanced security defenses against common threats such as automated bot registrations and credential stuffing attacks. ● Modernized architecture with strengthened security controls and greater application robustness. What is Discuz!

HTML, CSS,JS Chaining multiple bugs:

- Race Condition ⇒ Auth Bypass CVE-2026-49952
- CAPTCHA bypass using AI HTML, CSS,JS CVE-2026-49953
- RCE with administrator privileges CVE-2026-49954 Chaining multiple bugs:

□ 1. Race Condition ⇒ 3. Auth Bypass ⇒

⇒

⇒ □ 2. CAPTCHA bypass using AI HTML, CSS,JS

□ 4. RCE with administrator privileges Chaining multiple bugs:

□ Race Condition ⇒ Auth Bypass

HTML, □ CAPTCHA bypass using AI CSS,JS

□ RCE with administrator privileges RCE with administrator privileges □ Remote Code Execution (RCE) A Remote Code Execution (RCE) vulnerability may allow an attacker to execute arbitrary code on a remote system, without physical access, as if they were a local user.

⚙ Main Causes HTML, ● Insufficient input validation: malicious data interpreted as commands ● CSS,JS Insecure deserialization: serialized objects manipulated to execute code ● Memory corruption: e.g. errors such as buffer or integer overflow

□ Impact

● Complete system compromise ● Installation of malware or ransomware ● Theft of sensitive data ● Lateral movement within the network RCE with administrator privileges □ Example of RCE against a web application

HTML, CSS,JS RCE with administrator privileges □ Local File Inclusion (LFI) Local File Inclusion (LFI) is a web vulnerability that allows an attacker to force an application to load and execute unintended local files, interpreting them as source code ⇒ RCE ⚙ How it arises

● The application dynamically constructs paths to files to be included ● The attacker manipulates the path to cause arbitrary files to be executed

⚠ Difference from Directory Traversal (or Path Traversal)

● Directory Traversal → unauthorized access to files (for reading and/or writing) ● Local File Inclusion → inclusion and execution of files within the application flow

□ Impact of an LFI

● Reading sensitive files (e.g. configurations, passwords) ● Potential Remote Code Execution (RCE) on the server RCE with administrator privileges □ Example of Local File Inclusion (LFI) in PHP RCE with administrator privileges □ Example of Local File Inclusion (LFI) in PHP

How can we carry out a Remote Code Execution (RCE) attack starting from this LFI?

Esistono diverse possibilità, ad esempio se l'applicazione permette il caricamento di file (upload) da parte degli utenti (come l'upload dell'immagine di profilo), un attaccante potrebbe anzitutto caricare sul server un file malevolo contenente codice PHP arbitrario (solitamente una "webshell"), per poi includere lo stesso file nel flusso applicativo sfruttando la vulnerabilità LFI. RCE con privilegi di amministratore □ Esempio di Local File Inclusion (LFI) in PHP

First step of the attack:

The attacker creates a fake image file on their own machine containing PHP code similar to the following:

`<?php passthru($_GET['cmd']); ?>` The attacker saves this file as immagine.png and then uploads it to the server (inside the uploads subdirectory) through the application's upload functionality. RCE with administrator privileges □ Example of Local File Inclusion (LFI) in PHP

Second step of the attack:

The attacker exploits the LFI vulnerability to include the fake image in the application flow, thereby managing to remotely execute operating-system-level commands on the server: RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz! Bug reported

on 09/05/2026 here: gitee.com/Discuz/DiscuzX/issues/IJLFUW RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz! RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz! File: /source/app/admin/child/plugins/enable_disable.php

LFI at line 36: RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz! File: /source/app/admin/child/plugins/import.php The updatecache() function performs input validation and should (in theory) prevent Directory Traversal attacks... RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz!

The common_plugin table before the updatecache() function is invoked:

The common_plugin table after the updatecache() function has been invoked:

The moral of the story: by bypassing the call to the updatecache() function, it is possible to perform Directory Traversal attacks that lead to LFI and therefore RCE. RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz!

The updatecache() function invoked at line 112 performs input validation and should (in theory) prevent Directory Traversal attacks... But this can be “bypassed” thanks to the foreach loop at lines 85–91: because we control the \$pluginarray['var'] array, in a first step we can import a “fake plugin” with pluginvarid=123. Then we can import our “LFI plugin” in a second step, once again using pluginvarid=123. In this way, during the execution of line 90, an exception will be generated (by the DBMS) and the updatecache() function will not be invoked! RCE with administrator privileges □ Local File Inclusion (LFI) affecting Discuz! File: /source/app/admin/child/plugins/enable_disable.php

LFI at line 36: RCE with administrator privileges Exploit — Exploiting this LFI on Discuz! After obtaining access to the account of a user with administrator privileges, our exploit will perform the following steps:

1. Uploading the “stager” to the server through one of the upload features available in the administration panel (note: by “stager” we mean an image containing PHP code that will in turn write another PHP file to the server, namely our “webshell”, which will allow us to execute operating-system-level commands on the server);
2. Exploiting the LFI vulnerability to include and execute the “stager” in the application flow, thereby managing to write our “webshell” to the server;
3. Executing the “webshell” interactively.

RCE with administrator privileges Exploit - Exploiting this LFI on Discuz! Our “stager” will be the following PNG image, which was generated using the gen_plte_png.php script: RCE with administrator privileges Exploit - Exploiting this LFI on Discuz!

1. Uploading the “stager” to the server:

The “stager” will be uploaded inside the /data/attachment/common/cf/ directory RCE with administrator privileges Exploit - Exploiting this LFI on Discuz!

1. Exploiting the LFI to include our “stager”:

RCE with administrator privileges Exploit - Exploiting this LFI on Discuz!

1. Exploiting the LFI to include our “stager”:

RCE with administrator privileges Exploit - Exploiting this LFI on Discuz!

1. Esecuzione della “webshell” in modalità interattiva:

Sfruttare più bug in catena:

□ Race Condition $\Rightarrow$ Auth Bypass

HTML, □ Bypassing the CAPTCHA using AI CSS,JS

□ RCE with administrator privileges Bypassing the CAPTCHA using AI

Bug reported on 09/05/2026 here: gitee.com/Discuz/DiscuzX/issues/IJLFUW Bypassing the CAPTCHA using AI Discuz! uses CAPTCHA images both in the login endpoint and in the endpoint for registering new users, in order to prevent “automated” attacks through those endpoints...

HTML, CSS,JS Bypass del CAPTCHA usando l'IA ...e nel nostro exploit ci sarà bisogno sia di registrare un nuovo utente, sia di effettuare un tentativo di login con tale utente. Di conseguenza, è molto utile (anche se rimane comunque un optional) risolvere automaticamente questi CAPTCHA, cercando di creare un sistema o un programma in grado di riconoscere autonomamente il testo nelle immagini generate da Discuz! HTML, Vi svelo un segreto: per qualche strana ragione ero inizialmente CSS,JS convinto che per “vincere la corsa” (ovvero per sfruttare con successo la Race Condition di cui parleremo) fosse necessario bypassare i CAPTCHA, riconoscendo il testo presente nelle immagini nel minor tempo possibile... □

Spoiler: ciò non è necessario in realtà, ma solo “utile” per rendere il nostro exploit automatico al 100% □ Bypass del CAPTCHA usando l'IA Il bypass del CAPTCHA, ovvero il riconoscimento automatico del testo presente in queste immagini, è possibile attraverso un OCR (Optical Character Recognition, cioè “riconoscimento ottico dei caratteri”). Un OCR è un programma che esamina un'immagine e prova a riconoscere le lettere e/o i numeri presenti in essa.

Some OCR systems use neural networks, that is, artificial intelligence systems that learn from examples. These neural networks are first trained by showing them many images with the correct text (this set of examples is called a dataset). During this phase, they learn to recognize shapes and characters.

After training, the OCR can examine new images and try to read the text they contain automatically. Bypassing the CAPTCHA using AI Therefore, an OCR program based on a neural network seems perfectly suited to our needs...

HTML, CSS,JS

But a problem arises here: since I am not an expert in artificial intelligence (particularly Deep Learning), how can I create this OCR from scratch? Bypassing the CAPTCHA using AI HTML, CSS,JS Bypassing the CAPTCHA using AI ...so I fed ChatGPT about ten CAPTCHA images generated by Discuz! and asked whether it would be possible to create an OCR capable of automatically recognizing the characters in these images.

ChatGPT generated two Python scripts: train.py and infer.py. HTML, The first is used to train our OCR model, namely CSS,JS a neural network based on a CNN + LSTM + CTC

architecture, by providing as input a dataset containing a sufficiently large number of CAPTCHA images generated by Discuz!

The second, meanwhile, is used to recognize the text in the images by using the previously trained OCR model... Bypassing the CAPTCHA using AI ...I then wrote another Python script (make_dataset.py) to generate a dataset of more than 200,000 CAPTCHA images, which was then used by train.py to train the OCR model.

HTML, CSS,JS CAPTCHA bypass using AI □ make_dataset.py: In order for the script to work, the Discuz! source code must be modified as follows:

HTML, CSS,JS CAPTCHA bypass using AI Exploit - Registering a new user

HTML, CSS,JS CAPTCHA bypass using AI Exploit - Login attempt with the new user

HTML, CSS,JS Chaining multiple bugs:

□ Race Condition ⇒ Auth Bypass

HTML, □ CAPTCHA bypass using AI CSS,JS

□ RCE with administrator privileges Race Condition ⇒ Auth Bypass Bug reported on 09/05/2026 here: gitee.com/Discuz/DiscuzX/issues/IJLFUW

HTML, CSS,JS Race Condition ⇒ Auth Bypass Bug fixed on 09/05/2026 in commit 9962dad52c4c6999dabaf91ecd70377c680ff3c6:

HTML, CSS,JS Race Condition ⇒ Auth Bypass □ The Root of the Problem: Insecure Initialization The weakness in the code is located in the configuration file /config/config_ucenter.php, where the security constant UC_KEY is initialized by directly copying the value of "authkey"! Discuz! X3.5: Discuz! X5.0: HTML, CSS,JS Why is this a serious risk? In the specific case of this vulnerability, reusing a token or encryption key in different contexts (Cross-Context Token Reuse) lies at the heart of the problem and poses an extremely serious risk for fundamental cryptographic and logical design reasons... Race Condition ⇒ Auth Bypass □ The Leak Mechanism: Abuse of logging_more() The issue shifts to login handling. If the lssubmit parameter is sent during a login request, the application invokes the logging_ctl::logging_more() method:

HTML, CSS,JS

This method takes the username parameter directly from the GET request (and is therefore under the attacker's full control). A \$auth string is generated using the authcode() function, encrypting the username (and other data) with the "authkey" itself... Race Condition ⇒ Auth Bypass □ The Leak Mechanism: Abuse of logging_more() ...the generated \$auth code is then inserted into a JavaScript block (<script>) and returned directly to the user in the HTTP response:

HTML, CSS,JS

Poiché il sistema cifra l'username e restituisce il token cifrato valido, l'attaccante può utilizzare il campo username come vettore di injection, inserendo ad es. parametri di query string arbitrari. Payload di esempio: method=export&time=9999999999& Race Condition ⇒ Auth Bypass □ Conseguenze Critiche ● Generazione di Token Validi: l'attaccante ottiene

un “authcode” legittimo contenente i suoi comandi arbitrari, firmato direttamente dall'applicazione.

- Database Access: by using this valid “authcode,” the attacker can bypass authentication for the /api/db/dbbak.php script. HTML, CSS,JS

- Impact: through the /api/db/dbbak.php script, it becomes possible to export or import the platform’s entire database without possessing administrative credentials. Race Condition $\Rightarrow$ Auth Bypass $\square$ Critical Consequences: Access to /api/db/dbbak.php

Let us analyze the source code of this script, where it can be seen that the attacker controls both GET parameters, “code” and “apptype”:

HTML, CSS,JS

If \$apptype is equal to “discuzx”, the application will include the previously seen configuration file /config/config_ucenter.php, setting the UK_KEY constant equal to the value of “authkey”... Race Condition $\Rightarrow$ Auth Bypass $\square$ Critical Consequences: Access to /api/db/dbbak.php

HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass

HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass $\square$ Critical Consequences: Access to /api/db/dbbak.php

Example payload: method=export&time=9999999999&

Result:

```
$get['method'] =HTML, 'export'; CSS,JS $get['time'] = 9999999999;
```

And this will allow us to export the DB... Race Condition $\Rightarrow$ Auth Bypass $\square$ Critical Consequences: Access to /api/db/dbbak.php

Database export:

method=export&time=9999999999&

HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass $\square$ Critical Consequences: Access to /api/db/dbbak.php

Database export:

Thanks to lines of code 302–306, the common_member table will be the first to be exported! HTML, CSS,JS

And consequently, it will be the first to be deleted (and then recreated and therefore repopulated) during an import operation of that same SQL dump... Race Condition $\Rightarrow$ Auth Bypass

HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass

Why is this an incredibly lucky break?

Because this creates a so-called Race Condition $\square$ that we can exploit to our advantage to achieve our objective, namely obtaining access to an account with administrator privileges... HTML, CSS,JS

To understand how this is possible, we must first try to understand how Discuz!'s authentication mechanism works or, in this specific case, its cookie-based session management mechanism... Race Condition $\Rightarrow$ Auth Bypass The `discuz_application::_init_user()` method is responsible for “recognising” users through a session cookie that is decoded using the `authcode()` function:

HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass ...except that this time the `authcode()` function is invoked without the third parameter, namely without the encryption key, and in these cases `authcode()` uses a random encryption key, dynamically generated by the `discuz_application::_init_input()` method for each session: HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass ...therefore, somewhere in the Discuz! code, we need to find a call to the `authcode()` function in which we can control (even if only partially) the first parameter, while the second parameter must be 'ENCODE' and the third parameter must be absent, because this time we want a “session authcode” and not a simple “authcode”... HTML, Well, unfortunately, but also fortunately (again! ☺), I was CSS,JS able to find only one, and this call is located in the `/source/class/class_member.php` file inside the `logging_ctl::on_login()` method — but the problem is that it is inside an if branch, so its condition must be true — and this is precisely where our Race Condition arises... $\square$ Race Condition $\Rightarrow$ Auth Bypass ...at line 100, the `userlogin()` function is invoked and its result is stored in the `$result` variable. Then, at line 110, it checks whether the value of `$result['status']` is equal to -1, and if it is, it proceeds to generate a “session authcode”, constructed with a call to `authcode(username..., 'ENCODE')` at line 112 (which is then shown to the user at line 113): HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass This means that, once again, the username field can be used as an injection vector to obtain a valid “session authcode” (signed by Discuz!), which can be reused as a session cookie to impersonate an administrator user...

However, we need `$result['status']` to be equal to -1 ... HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass ...and, as it happens, `$result['status']` will be equal to -1 if and only if the supplied username is present in the `ucenter_members` table, but not in the `common_member` table...

Well, do you remember that, in addition to exporting, we can also import a previously created SQL dump?! HTML, CSS,JS During an import operation of our SQL dump, the `common_member` table will be the first to be deleted! Therefore, there will be a brief moment $\square$ in which the username registered by the attacker will be present in the `ucenter_members` table, but not in the `common_member` table... Race Condition $\Rightarrow$ Auth Bypass ...this will set `$result['status']` to -1 if we send a login request before our username is also removed from the `ucenter_members` table — and here is our infamous Race Condition $\square$ — thus entering the if branch at line 110 and returning to the attacker a “session authcode” constructed with their username! $\square$ HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass Therefore, before attempting to “win the race” (that is, the Race Condition $\square$), all that remains for the attacker to do is register a new user whose username is a string similar to the following:

e8c61d09a2af09c1bd4088a1252a693d 1 RANDOM HTML, CSS,JS The first part represents the MD5 hash of a random password associated with the user (corresponding to the

password column of the common_member table), followed by a tab, then the ID associated with the user, and then another tab followed by a random string... The MD5 hash associated with the administrator user can easily be obtained during the database export!

Race Condition $\Rightarrow$ Auth Bypass

Once this is done, we can try to “exploit” our wonderful Race Condition $\square$ in the hope of “winning” it, which would give us access to a “session authcode” that we can exploit by using it as a session cookie to impersonate the administrator user, then reset their password, access the Discuz! administrative panel, and finally “exploit” the RCE vulnerability that we saw earlier! HTML, CSS,JS Race Condition $\Rightarrow$ Auth Bypass Exploit - Exploiting the Race Condition:

HTML, CSS,JS Brief exploit analysis & HTML, CSS,JS Disclosure Timeline Exploit - Overview:

HTML, CSS,JS Exploit - Main function:

HTML, CSS,JS Exploit - Main function:

HTML, CSS,JS Exploit - Infographic:

HTML, CSS,JS $\square$ Responsible Disclosure Timeline: 27/04/2026 $\square$ Initial contact with the developers through messages on Gitee.com — no response

$\square$ Email to the vendor (admin@discuz.vip and security@tencent.com) — no response

07/05/2026 $\square$ Opened issue #IJLFUW on <https://gitee.com/Discuz/DiscuzX>

09/05/2026 $\square$ Shared technical details of the vulnerabilities in issue #IJLFUW HTML, $\square$ Fixed the Race Condition $\Rightarrow$ Auth Bypass vulnerability on Gitee.com (commit 9962da...)

CSS,JS 10/05/2026 $\square$ Released a new version (Discuz_X5.0_20260510.zip) containing the vulnerability patch

09/06/2026 $\square$ CVEs requested $\rightarrow$ $\square$ CVE-2026-49952, CVE-2026-49953, and CVE-2026-49954 assigned

13/06/2026 (hackmeeting) $\square$ Public disclosure and presentation of the research

Archived from https://docs.google.com/presentation/d/178bNsA_BtCQzYK02sfhkis0xkDst-wPadT3Y_YMPZWQ.

Rendered offline from the local Markdown copy.